Problem Statement :– Text Summarization.

The goal is to build a system that generates a summary of long text documents while preserving the most important information and overall meaning.
Text summarization helps in extracting the most relevant sentences from a document using statistical techniques such as Term Frequency (TF) and Inverse Document Frequency (IDF).

Process Steps involved to create the text summary :-

1. **Tokenize sentences**

   Tokenization means splitting text into smaller pieces (tokens). When we tokenize sentences, we split a paragraph into separate sentences using rules like punctuation, capitalization, abbreviations, etc.
   Example "I love data science. It is fascinating! Do you agree?"
   Sentence tokenization output:
   1. *I love data science.*
   2. *It is fascinating!*
   3. *Do you agree?*

   Code Snippet:

```python
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
import string
import math

# **********************Tokenize Sentences**********************
text = """Artificial Intelligence is transforming many industries.
Machine learning helps computers learn from data.
AI is used in healthcare, finance, and automation."""

sentences = sent_tokenize(text)
#print(sentences)
```

2. **Create frequency matrix of words in each sentence**

   A Frequency Matrix is a structured table that shows how many times each word appears in each document or sentence. It is used in Natural Language Processing (NLP), text mining, and machine learning.
   Example:

Word	Sentence 1	Sentence 2	Sentence 3
data	1	0	0
science	1	0	0
AI	0	2	1

Code Snippet:

```python
# **********************Create Frequency Matrix***********************
print("Create Frequency Matrix")
stopWords = set(stopwords.words("english"))
freq_matrix = {}
for sent in sentences:
    freq_table = {}
    words = word_tokenize(sent.lower())
    #print("In Sentences loop")
    for word in words:
        #print("In Word loop:", word )
        if word in stopWords or word in string.punctuation:
            continue
        if word in freq_table:
            freq_table[word] += 1
        else:
            freq_table[word] = 1

    freq_matrix[sent[:15]] = freq_table
    #print(freq_matrix)
```

3. Calculate Term Frequency and Generate matrix

Term Frequency (TF) is a basic concept in Natural Language Processing (NLP) that measures how often a word appears in a document. It helps convert text into numerical values so algorithms can understand and compare documents.

Formula:- TF = Number of times term t appears in a document / Total number of terms in the document

Example:- "Data science is fun and data drives decisions."

Total words = 7

"data" appears = 2 times

TF(data) = 2/7 = 0.285

Code Snippet:

```python
# **********************Calculate Term Frequency (TF)***********************
print("Calculate Term Frequency (TF)")
tf_matrix = {}
for sent, freq_table in freq_matrix.items():
    tf_table = {}
    total_words = sum(freq_table.values())
    #print("Total Words:", total_words)
    for word, count in freq_table.items():
        tf_table[word] = count / total_words

    tf_matrix[sent] = tf_table

```

4. Create a table for documents per words

A Document-Per-Word Table is a simple NLP table that shows which documents contain which words. This ensures whether the word appears at all (yes/no).

Example: -

"AI is powerful"

"AI and data"

"Data is important"

Word	Doc1	Doc2	Doc3
ai	1	1	0
is	1	0	1
powerful	1	0	0
and	0	1	0
data	0	1	1
important	0	0	1

Code Snippet:

```
# **********************Create Document Per Word Table**********************
word_per_doc = {}
print("Create Document Per Word Table")
for sent, freq_table in freq_matrix.items():
    for word in freq_table:
        if word in word_per_doc:
            word_per_doc[word] += 1
        else:
            word_per_doc[word] = 1
```

5. Calculate IDF and generate matrix

IDF (Inverse Document Frequency) measures how important a word is across multiple documents. It helps reduce the weight of very common words (like the, is, and) and increases the weight of rare, meaningful words.

Formula: - $IDF(t) = \log(N/ df_t)$

N = total number of documents

df_t = number of documents that contain term t

Example: -

"AI is powerful"

"AI and data"

"Data is important"

Word	df (documents containing the word)
ai	2
is	2
powerful	1

and	1
data	2
important	1

Code Snippet:

```python

# **********************Calculate IDF Matrix**********************
print("Calculate IDF Matrix")
idf_matrix = {}
total_sentences = len(freq_matrix)

for sent, freq_table in freq_matrix.items():
    idf_table = {}

    for word in freq_table:
        idf_table[word] = math.log10(total_sentences / float(word_per_doc[word]))

    idf_matrix[sent] = idf_table
#print("IDF Matrix:", idf_matrix)
```

6. Calculate TF-IDF and generate matrix

Term Frequency - Inverse Document Frequency

It is a numerical measure to show how important a word is in a document relative to a collection of documents.

Formula:- TF-IDF(t,d)=TF(t,d)×IDF(t)

TF → importance inside a document

IDF → importance across documents

TF-IDF → final weighted score showing which words are most meaningful

High values → important, rare, meaningful

Low values → common or irrelevant words

Code Snippet:

```python
# **********************Calculate TF-IDF Matrix**********************
print("Calculate TF-IDF Matrix")
tf_idf_matrix = {}

for sent in tf_matrix:
    tf_idf_table = {}

    for word in tf_matrix[sent]:
        tf_idf_table[word] = tf_matrix[sent][word] * idf_matrix[sent][word]

    tf_idf_matrix[sent] = tf_idf_table
#print("TF-IDF Matrix:", tf_idf_matrix)
```

7. Score the sentences

Each sentence gets a score based on the average TF-IDF values of its words.
Sentence score = average TF-IDF score of all words in that sentence.

Code Snippet:

```
# **********************Score the Sentences**********************
print("Score the Sentences")
sentence_scores = {}
for sent, tfidf_table in tf_idf_matrix.items():
    total_score = sum(tfidf_table.values())
    sentence_scores[sent] = total_score / len(tfidf_table)
#print("Sentence Scores:", sentence_scores)
```

8. Find the threshold

The threshold is calculated as the average score of all sentences.
Threshold = average of sentence scores

Code Snippet:

```
# **********************Find the Threshold**********************
print("Find the Threshold")
sum_scores = sum(sentence_scores.values())
threshold = sum_scores / len(sentence_scores)
#print("Threshold:", threshold)
```

9. Generate the summary

Sentences with scores greater than the threshold are included in the summary.

Code Snippet:

```
# **********************Generate summary**********************
print("Generate summary")
summary = ""

for sent in sentences:
    if sentence_scores[sent[:15]] >= threshold:
        summary += " " + sent

print("Summary:-",summary)
```

10. Tool used and reasons to use this specific tool
    1. Python 3
        a. Simple and easy to understand syntax
        b. Large ecosystem of NLP and machine learning libraries

c. Supports text processing and data analysis
d. Faster development and easy implementation
e. Strong community support

Because of these advantages, Python is widely used for text summarization and other NLP applications.

2. **NLTK Toolkit** – NLTK simplifies many Natural Language Processing tasks such as:

- Sentence Tokenization – splitting text into sentences
- Word Tokenization – splitting sentences into words
- Stopword Removal – removing common words like is, the, and
- Text preprocessing – preparing text for analysis

These features make it very useful for building text summarization systems.

3. **IDE / Text Editor (VS Code)** – An IDE helps in:

- Writing and editing code easily
- Debugging errors
- Running Python programs
- Managing project files

It improves development speed and productivity.

11. Code Snippet

TextSummarization.py

TextSummarization.py > ...

```python
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
import string
import math

# **********************Tokenize Sentences**********************
text = """Artificial Intelligence is transforming many industries.
Machine learning helps computers learn from data.
AI is used in healthcare, finance, and automation."""

sentences = sent_tokenize(text)
#print(sentences)

# **********************Create Frequency Matrix**********************
print("Create Frequency Matrix")
stopWords = set(stopwords.words("english"))
freq_matrix = {}
for sent in sentences:
    freq_table = {}
    words = word_tokenize(sent.lower())
    #print("In Sentences loop")
    for word in words:
        #print("In Word loop:", word )
        if word in stopWords or word in string.punctuation:
            continue
        if word in freq_table:
            freq_table[word] += 1
        else:
            freq_table[word] = 1

    freq_matrix[sent[:15]] = freq_table
    #print(freq_matrix)
```

```python
# **********************Calculate Term Frequency (TF)**********************
print("Calculate Term Frequency (TF)")
tf_matrix = {}
for sent, freq_table in freq_matrix.items():
    tf_table = {}
    total_words = sum(freq_table.values())
    #print("Total Words:", total_words)
    for word, count in freq_table.items():
        tf_table[word] = count / total_words

    tf_matrix[sent] = tf_table

# **********************Create Document Per Word Table**********************
word_per_doc = {}
print("Create Document Per Word Table")
for sent, freq_table in freq_matrix.items():
    for word in freq_table:
        if word in word_per_doc:
            word_per_doc[word] += 1
        else:
            word_per_doc[word] = 1

# **********************Calculate IDF Matrix**********************
print("Calculate IDF Matrix")
idf_matrix = {}
total_sentences = len(freq_matrix)

for sent, freq_table in freq_matrix.items():
    idf_table = {}

    for word in freq_table:
        idf_table[word] = math.log10(total_sentences / float(word_per_doc[word]))

    idf_matrix[sent] = idf_table
#print("IDF Matrix:", idf_matrix)
```

```python
# **********************Calculate TF-IDF Matrix**********************
print("Calculate TF-IDF Matrix")
tf_idf_matrix = {}

for sent in tf_matrix:
    tf_idf_table = {}

    for word in tf_matrix[sent]:
        tf_idf_table[word] = tf_matrix[sent][word] * idf_matrix[sent][word]

    tf_idf_matrix[sent] = tf_idf_table
#print("TF-IDF Matrix:", tf_idf_matrix)
# **********************Score the Sentences**********************
print("Score the Sentences")
sentence_scores = {}
for sent, tfidf_table in tf_idf_matrix.items():
    total_score = sum(tfidf_table.values())
    sentence_scores[sent] = total_score / len(tfidf_table)
#print("Sentence Scores:", sentence_scores)
# **********************Find the Threshold**********************
print("Find the Threshold")
sum_scores = sum(sentence_scores.values())
threshold = sum_scores / len(sentence_scores)
#print("Threshold:", threshold)
# **********************Generate summary**********************
print("Generate summary")
summary = ""

for sent in sentences:
    if sentence_scores[sent[:15]] >= threshold:
        summary += " " + sent

print("Summary:-",summary)
```

12. Final output results and analysis of results

Result:

```python
    idf_matrix[sent] = idf_table
#print("IDF Matrix:", idf_matrix)
# **********************Calculate TF-IDF Matrix**********************
print("Calculate TF-IDF Matrix")
tf_idf_matrix = {}

for sent in tf_matrix:
    tf_idf_table = {}

    for word in tf_matrix[sent]:
        tf_idf_table[word] = tf_matrix[sent][word] * idf_matrix[sent][word]

    tf_idf_matrix[sent] = tf_idf_table
#print("TF-IDF Matrix:", tf_idf_matrix)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\ssarode2\Desktop\Shruti's personal\MTECH Docs\SEM 2\CS> & C:/Users/ssarode2/AppData/Local/Programs/Python/Python313/p
/Desktop/Shruti's personal/MTECH Docs/SEM 2/CS/TextSummarization.py"
Create Frequency Matrix
Calculate Term Frequency (TF)
Create Document Per Word Table
Calculate IDF Matrix
Calculate TF-IDF Matrix
Score the Sentences
Find the Threshold
Generate summary
Summary:-  Artificial Intelligence is transforming many industries. AI is used in healthcare, finance, and automation.
PS C:\Users\ssarode2\Desktop\Shruti's personal\MTECH Docs\SEM 2\CS>
```

Input Text:

Artificial Intelligence is transforming many industries. Machine learning helps computers learn from data. AI is used in healthcare, finance, and automation.

Generated Summary:

Artificial Intelligence is transforming many industries. AI is used in healthcare, finance, and automation.

After applying the summarization process (tokenization, TF, IDF, TF-IDF calculation, sentence scoring, and threshold selection), the algorithm identifies the most significant sentences in the document and combines them to form the final summary.
By using the TF-IDF method, the system assigns importance scores to words and sentences, allowing it to identify sentences that carry the most meaningful information.

Advantages Observed in the Result:

- Efficient information extraction from long documents
- Improved readability by reducing unnecessary content
- Faster understanding of large texts
- Useful for news summarization, document analysis, and research papers

Limitations of the Result

- Although the model produces useful summaries, some limitations exist:
- The system only extracts sentences, it does not generate new ones.
- Some summaries may lack smooth flow between sentences.
- Important context may sometimes be missed if sentence scoring is imperfect.

- These limitations occur because the approach uses extractive summarization techniques.

Conclusion

The implemented text summarization model successfully generates a concise summary of long text documents using the TF-IDF technique. The results show that the system can effectively identify and extract important sentences, reducing document size while maintaining the key message of the original content.